TERSE

Token-Efficient Recursive Serialization Encoding

Format Specification Draft 0.6

Author: Rudson Kiyoshi Souza Carvalho

TERSE is a text-based data serialization format designed to represent the complete JSON data model with substantially fewer tokens, making it significantly more cost-efficient for use as input to Large Language Models (LLMs). Unlike existing token-efficient formats such as TOON, TERSE fully supports the entire JSON value space, including arbitrarily nested objects, heterogeneous arrays, all primitive types, and empty collections.

Status

Field	Value
Status	Draft — not yet ratified
Version	0.6.0
Date	March 2026
Authors	Open for contribution
License	CC BY 4.0

1. Motivation

JSON (JavaScript Object Notation) is the de facto standard for data interchange. Its design priorities are human readability and machine parseability, both of which are well served by its explicit syntax: quoted keys, braces, brackets, and commas.

However, LLMs process text as a sequence of tokens. In modern BPE tokenizers, every structural character in JSON — every quotation mark, every brace, every repeated key name — occupies one or more tokens. For a model being billed per

input token, JSON's structural verbosity is a direct cost with no informational return.

Existing alternatives address only a subset of the problem:

- **TOON** (Token-Oriented Object Notation) reduces tokens for flat, uniform arrays but does not support nested objects or heterogeneous structures.
- **YAML** is more verbose than JSON for structured data due to its dash-indented list syntax.
- **CSV** has no concept of nesting, types, or null values.
- **Binary formats** (Protobuf, MessagePack) require pre-defined schemas and cannot be consumed directly by LLMs as text.

TERSE addresses the gap: a text format that covers the full JSON value space while minimizing token overhead. It achieves this through five principles:

1. Bare strings — identifiers and common values require no quotation marks.
2. Compact primitives — null, true, false are single characters.
3. Implicit delimiters — spaces separate values; no trailing commas.
4. Schema arrays — uniform arrays declare fields once, not per row.
5. Recursive structure — all constructs may be nested arbitrarily.

---

2. Terminology

The key words MUST, MUST NOT, REQUIRED, SHALL, SHOULD, MAY are used as defined in RFC 2119.

Term	Definition
Document	A TERSE document is a sequence of top-level key-value pairs, one per logical line.
Value	Any of: null, boolean, number, string, object, array, or schema-array.
Bare string	A string that satisfies the safe-identifier grammar and requires no quotation marks.
Quoted string	A string enclosed in double quotation marks, using JSON string escape sequences.
Schema array	An array whose entries all share the same set of keys, declared with a #[fields] header.
Block form	An object or array written across multiple lines using indentation.
Inline form	An object or array written on a single line within { } or [ ] delimiters.

3. Lexical Rules

3.1 Character Encoding

TERSE documents MUST be encoded as UTF-8. There is no BOM requirement.

3.2 Whitespace

Within a line, horizontal whitespace (space U+0020) separates tokens. Tab characters (U+0009) MUST NOT be used as value separators or for indentation within TERSE documents — parsers MUST reject documents that use tabs for indentation.

Block indentation MUST be exactly two spaces (U+0020 U+0020) per nesting level. One space and three spaces are both invalid. Mixing tab and space indentation is forbidden.

A newline is U+000A (LF). CRLF sequences (U+000D U+000A) MUST be normalized to LF before parsing.

3.3 Comments

Lines whose first non-whitespace characters are // are comments and MUST be ignored by parsers. Inline comments are not permitted in this version of the specification.

```
// This is a comment
name: my-app   // NOT valid — inline comments are forbidden
```

3.4 Reserved Tokens

The following single-character and two-character sequences are reserved as literals and MUST NOT appear as bare strings:

Token	Meaning	Example
`~`	null	`active: ~`
`T`	true (boolean)	`verified: T`
`F`	false (boolean)	`debug: F`
`{}`	empty object	`meta: {}`
`[]`	empty array	`tags: []`

> *A string whose value is literally "T", "F", or "~" MUST be quoted: name: "T"*

3.5 Safe Identifier Grammar

A bare string is any sequence of characters matching the following ABNF-aligned grammar. The character set was expanded in Draft 0.2 to include . and / so that domain names, file paths, and semver strings do not require unnecessary quoting.

Note: ALPHA in this grammar means ASCII letters A–Z and a–z (U+0041–U+005A, U+0061–U+007A) only. Unicode letters outside ASCII are NOT safe-id characters and MUST be placed in quoted strings.

```
; Revised in Draft 0.2 — expanded safe-char set
safe-start = ALPHA / "_" / "." / "/"
safe-char  = ALPHA / DIGIT / "-" / "_" / "." / "/" / "@"
safe-id    = safe-start *safe-char

; A safe-id MUST NOT be identical to a reserved token:
;   "T"  "F"  "~"  "{}"  "[]"
; A safe-id MUST NOT match the number grammar.

; Examples that are now bare (no quotes needed):
;   api.example.com   /usr/local/bin   v2.1.0   alice@co.com
```

If a string value does not satisfy the safe-id grammar, it MUST be represented as a quoted string.

---

4. Value Types

4.1 Null

The null value is represented by the single character ~.

```
notes: ~
```

4.2 Boolean

The boolean values true and false are represented by T and F respectively.

```
active: T
debug: F
```

4.3 Number

Numbers follow the JSON number grammar: an optional leading minus sign, an integer part, an optional decimal part, and an optional exponent. No leading zeros are permitted except for the value 0 itself.

Parser precedence rule: when a token could be interpreted as either a number or a safe-id, it MUST be interpreted as a number. Parsers MUST apply number grammar matching before safe-id matching. Consequently, values such as 1e3, -5,

and 3.14 are always numbers, never strings. To represent these as strings, they MUST be quoted.

```
port: 3000       // number
ratio: 1.5e-3    // number — scientific notation
score: -4.2      // number
label: "1e3"    // string — quoted to force string type
code: "404"     // string — quoted to force string type
```

4.4 String

4.4.1 Bare Strings

If a string satisfies the safe-id grammar, a conforming serializer MUST write it without quotes. Parsers MUST accept both bare and quoted representations of the same string value.

```
env: production
host: api.example.com
```

4.4.2 Quoted Strings

Strings that do not satisfy the safe-id grammar MUST be enclosed in double quotation marks and MUST use JSON string escape sequences (\n, \t, \\, \", \uXXXX, etc.).

```
message: "Hello, world!"
path: "C:\\Users\\Alice"
truth: "T"      // quoted because T alone means boolean
```

4.5 Object

4.5.1 Inline Form

An object with a short, flat representation MAY be written inline using { key:value key2:value2 } syntax. No commas are used. Whitespace separates key-value pairs.

```
author: {name:Alice email:"alice@co.com"}
```

> *Inline parsing rule: parsers MUST use a greedy, depth-aware tokenizer when parsing inline objects. A { token always opens a nested object scope; the matching } closes it. Whitespace between key:value pairs is consumed as a delimiter. Example: {a:1 b:{c:2 d:3} e:4} is unambiguous — b's value is the nested object {c:2 d:3}.*

4.5.2 Block Form

An object that does not fit on a single line, or whose values are themselves multi-line, MUST use block form. The opening { and closing } each occupy their own line at the current indentation level. Key-value pairs inside the block are indented by two spaces relative to the parent key — that is, at the same indentation level as the { brace itself.

```
db:
  {
  host: db.internal    // same indent level as {
  port: 5432
  pool: {min:2 max:10}
```

```
  }
```

> *The content is NOT indented relative to { — both { and the key-value pairs share the same two-space indent from the parent key. This keeps visual alignment predictable and consistent with how block arrays work.*

4.5.3 Empty Object

An empty object is written as {}.

4.6 Array

4.6.1 Inline Form

An array whose total serialized length fits on one line MAY be written as [v1 v2 v3]. Values are whitespace-separated; no commas.

```
tags: [web typescript spa]
scores: [95 87 92]
flags: [T F T ~]
```

4.6.2 Block Form

An array that does not fit on a single line uses block form: [on the first line, each value indented,] on the last line.

```
items:
  [
  {sku:A1 qty:2 price:9.99}
  {sku:B3 qty:1 price:24.50}
  ]
```

4.6.3 Schema Array

When all elements of an array are objects that share the same set of keys, a conforming serializer MUST represent the array as a schema array. This is the primary source of token savings for tabular data.

Syntax:

```
#[field1 field2 ... fieldN]
  value1 value2 ... valueN
  value1 value2 ... valueN
```

Rules:

- The header line begins with #[and ends with].
- Field names are whitespace-separated bare strings.
- Each data row is indented by exactly two spaces.
- Values on each row correspond positionally to the declared fields.
- String values containing whitespace, reserved characters, or any character outside the safe-id grammar MUST be quoted within schema array rows. This is strictly required — an unquoted multi-word string would be misparse as multiple positional values.
- A missing or null field value in a row MUST be represented as ~ (not an empty token or omitted position). Every row MUST supply exactly N values for N declared fields.

```
// Correct quoting in schema arrays
users:
  #[id name role active score]
  1 Alice admin T 98.5
  2 Bob editor T 87.2
  3 "Casey Lee" viewer F 72.0    // quoted: contains space
  4 "Maria-José" ~ ~ 61.0        // quoted: hyphen at start; null role

// INVALID — unquoted value with space creates wrong column count
  3 Casey Lee viewer F 72.0       // WRONG: parsed as 6 values, not 5
```

4.6.4 Empty Array

An empty array is written as [].

---

5. Document Structure

A TERSE document at the top level is a sequence of key-value pairs, not wrapped in an outer object. Each pair follows the form:

```
key: value
// or, for multi-line values:
key:
  value
```

Semantically, a TERSE document is equivalent to a JSON root object. A conforming parser MUST produce a JSON object (or its language-native equivalent) as the result of parsing a document, with each top-level key-value pair becoming a member of that object. A TERSE document with no key-value pairs is equivalent to the empty JSON object {}.

> *This means TERSE cannot represent a top-level JSON array or primitive directly. To serialize a top-level array, it MUST be wrapped in a named key: e.g. data: [1 2 3]*

A complete example:

```
// App configuration
name: my-app
version: 2.1.0
private: T
author: {name:Alice email:"alice@co.com"}
scripts: {dev:vite build:"vite build" test:vitest}
config:
  {
  port: 3000
  debug: F
  tags: [web typescript]
  }
dependencies:
  {
  react: "^18.2.0"
  zustand: "^4.4.1"
  }
```

---

6. Encoding Rules (Serializer Behavior)

A conforming TERSE serializer MUST apply the following rules when converting from a native data structure or JSON:

1. Output ~ for null.
2. Output T for true, F for false.
3. Output numbers using the shortest representation that round-trips.
4. Output bare strings for values satisfying the safe-id grammar; quoted strings otherwise.
5. Output {} for empty objects; [] for empty arrays.
6. Attempt inline form first for objects and arrays; fall back to block form if the result exceeds the line length limit.
7. Use schema array form for arrays of two or more uniform objects.
8. Keys at the document level are written without an enclosing { }.

> *Serializers MUST NOT abbreviate key names. TERSE does not define a key-shortening mechanism; that is left to the application layer.*

---

7. Decoding Rules (Parser Behavior)

A conforming TERSE parser MUST reconstruct a lossless JSON-equivalent data structure:

- ~ decodes to JSON null.
- T decodes to JSON true; F decodes to JSON false.
- Tokens matching the number grammar decode to JSON numbers.
- Bare strings decode to JSON strings.
- Quoted strings decode to JSON strings, with escape sequences resolved.
- { ... } decodes to a JSON object.
- [...] decodes to a JSON array.
- #[fields] rows decode to an array of JSON objects.

Parsers MUST be tolerant of trailing whitespace and blank lines. Parsers MUST raise an error on malformed escape sequences inside quoted strings.

Duplicate keys: if an object contains two or more entries with the same key, the behavior is implementation-defined. Parsers SHOULD raise a parse error on duplicate keys. Serializers MUST NOT produce duplicate keys.

---

8. Comparison with Related Formats

Format	Characteristics
JSON	Universal. Full spec coverage. Verbose syntax. Native parsers in every language.
TOON	Token-efficient for flat arrays. No support for nested objects or mixed arrays.
YAML	Full JSON spec coverage (JSON is a YAML subset). Human-friendly authoring. ~20–24% fewer tokens than JSON. Parser spec is complex with known edge cases. Not optimized for LLM token efficiency.
CSV	Extremely compact for tabular data. No types, no nesting, no null.
Protobuf	Binary. Requires compiled schema. Cannot be consumed directly by LLMs as text.
TERSE	Token-efficient. Full JSON spec coverage. Text-based. LLM-native. ~24% fewer tokens than YAML, ~47% fewer than JSON.

8.1 JSON Spec Coverage

Feature	JSON	TERSE	YAML	TOON	CSV
null	✓	✓	✓	✗	✗
boolean	✓	✓	✓	✗	✗
number	✓	✓	✓	✓ (numeric only)	✗
string	✓	✓	✓	✓	✓ (no escaping)
nested objects	✓	✓	✓	✗	✗
heterogeneous arrays	✓	✓	✓	✗	✗
uniform arrays	✓	✓	✓	✓	✓
empty collections	✓	✓	✓	✗	✗
LLM-optimized tokens	✗	✓	✗	✗	✗
Simple parser spec	✓	✓	✗	✓	✓

> *YAML is the only alternative that covers the full JSON data model. In fact, JSON is formally a subset of YAML — every valid JSON document is also valid YAML. However, YAML was designed for human authoring, not token efficiency. Its parser specification is notoriously complex (YAML 1.2 has well-documented edge cases and ambiguities), while TERSE is designed to be simple and predictable to parse. The practical*

> *distinction: use YAML when humans write the data, use TERSE when machines serialize data for LLM consumption.*

8.2 Token Efficiency vs YAML

YAML is often proposed as a more readable and compact alternative to JSON for LLM input. While YAML does reduce tokens compared to JSON (~20–24% savings), it remains significantly more verbose than TERSE for two structural reasons: arrays require one line per item with a leading dash token (- value), and boolean and null values use full words (true, false, null) rather than single characters.

Document Type	JSON tokens	YAML tokens	TERSE tokens	YAML vs JSON	TERSE vs JSON
App config	~85	~65	~45	-24%	-47%
Nested object	~110	~82	~55	-25%	-50%
Flat array (5 rows)	~120	~95	~58	-21%	-52%
Deep nesting	~180	~140	~105	-22%	-42%

> *Token counts are approximate estimates using cl100k_base on representative examples. YAML optimizes for human authoring; TERSE optimizes for LLM consumption. Both goals are valid — they are simply different.*

9. Token Efficiency

The following measurements were obtained using the cl100k_base tokenizer (OpenAI's BPE tokenizer, used by GPT-3.5 and GPT-4) on representative document types. Character counts are provided as an additional proxy applicable across other tokenizers.

Document Type	TERSE vs JSON	YAML vs JSON	TOON vs JSON	CSV vs JSON
App config (package.json style)	~35–45%	~20–24%	~15–20%	N/A
User list (flat, 5 records)	~40–55%	~18–22%	~40–55%	~40–55%
Structured logs (array, 5 entries)	~35–45%	~18–22%	~35–45%	~35–45%
Product catalog (nested + array)	~30–40%	~20–25%	~10–15%	N/A
Complex order (deep nesting)	~25–35%	~20–25%	~5–10%	N/A
AI tool call payload (deep)	~20–30%	~18–22%	~5–10%	N/A

> *TOON and CSV savings are blank (N/A) for nested/complex types because those formats cannot represent those structures. YAML covers all types but uses full-word booleans (true/false) and per-item array syntax (- value), limiting its token savings to ~18–25%. TERSE maintains the highest savings across all document types.*

Reading the table: YAML is a genuine improvement over JSON for LLM input — roughly 20–24% savings — and covers the full data model. However, TERSE goes further: ~47% savings on average, because it was designed specifically for token efficiency rather than human authoring. The key differences are compact primitives (T/F/~ vs true/false/null), inline arrays ([a b c] vs one line per item), and inline objects ({k:v} vs indented blocks). For real-world LLM workloads — tool call payloads, RAG context, agent state, API responses — those differences compound at scale.

---

10. Conformance

10.1 Conforming Serializer

A conforming TERSE serializer MUST:

- Produce output that is parseable by any conforming TERSE parser.
- Represent all JSON value types correctly.
- Use schema array form for uniform object arrays of two or more elements.
- Produce lossless round-trip results: parse(serialize(x)) deep-equals x for all valid inputs.

10.2 Conforming Parser

A conforming TERSE parser MUST:

- Accept all output produced by any conforming serializer.
- Produce a data structure equivalent to the JSON representation of the TERSE input.
- Raise a parse error on malformed input rather than silently producing incorrect output.

10.3 Test Suite

A reference test suite is maintained alongside this specification at https://github.com/terse-format/spec (URL provisional — to be confirmed before v1.0 ratification). Implementations SHOULD pass all normative tests before claiming conformance.

---

11. Open Questions (Draft Items)

The following issues are open for discussion and will be resolved before the 1.0 ratification:

1. **Key ordering:** Should TERSE mandate key ordering (e.g., alphabetical) to improve determinism across serializers? JSON does not require ordering.
2. **Multi-line strings:** A block string syntax (similar to YAML's | literal block) may be useful for long text values. Not yet specified.
3. **Schema array heterogeneity:** The current spec requires all rows to have the same fields. A future version may allow optional fields via a sentinel.
4. **Comment preservation:** Comments are stripped by the parser in this version. A future version may define a comment-preserving AST.
5. **Streaming:** Streaming serialization and parsing are implementation concerns not yet addressed by this spec.
6. **Binary values:** Base64-encoded binary blobs are currently treated as ordinary strings. A dedicated literal type may reduce ambiguity.

---

12. Design Rationale

Several design decisions in TERSE deserve explicit justification:

Why T/F/~ instead of true/false/null?

These are the most common boolean and null values in real-world data. Replacing six-character tokens with single characters yields meaningful savings at scale and does not reduce readability — any reader familiar with the spec immediately understands them. The risk of collision with string values is addressed by the quoting requirement ("T" for the string T).

Why spaces instead of commas as value separators?

Commas are structural noise in the context of LLM token consumption. Modern tokenizers often attach commas to adjacent words, creating inconsistent tokenization. Space-separated values produce more predictable token boundaries.

Why not abbreviate key names automatically?

Automatic key abbreviation would break the readability and debuggability of the format. It would also require a separate schema to decode the abbreviations, reintroducing the overhead it sought to avoid. Applications that benefit from key abbreviation should define their own short key names and use TERSE for value encoding.

Why use #[ ] for schema arrays instead of TOON's syntax?

TOON uses key{count}: as its schema array delimiter, which carries implicit information (the count) that duplicates what can be inferred from the data.

TERSE's #[fields] header makes the schema explicit and readable while being shorter. The # prefix visually distinguishes schema headers from data rows.

---

13. Security Considerations

Because TERSE is designed for use as input to autonomous LLM agents and production pipelines, implementations MUST address the following risks.

13.1 Context Injection and Delimiter Hijacking

TERSE uses spaces and newlines as primary delimiters. If unsanitized user-controlled strings are inserted into a TERSE document as bare strings, an attacker may inject structural characters that alter the parse tree.

- **Mitigation:** Conforming serializers MUST quote any string that contains reserved or structural characters: { } [ ] : # ~ \ or any whitespace or newline sequence. The safe-id grammar enforces this when applied correctly.
- **Recommendation:** Applications SHOULD treat all externally-sourced string values as potentially hostile and pass them through the serializer rather than embedding them manually.

13.2 Semantic Ambiguity with Reserved Tokens

The tokens T, F, and ~ are interpreted as typed primitives. An attacker who can influence a value field may inject ~ into a field expected to hold a non-null string, causing downstream null-pointer or type errors in the consuming application.

- **Rule:** Parsers MUST surface typed values as-is (boolean, null) and not silently coerce them. Applications performing Type-Safe Decoding SHOULD validate decoded values against an expected schema before use.
- **Example:** A status field decoded as null (~) where a string was expected SHOULD raise a validation error, not be silently passed downstream.

13.3 Canonical Representation and Signature Stability

Because TERSE does not mandate key ordering, two serializations of the same logical document may differ in byte content. This is problematic for cryptographic signatures, audit logs, and deterministic hashing.

- **Recommendation:** Serializers intended for security-sensitive contexts SHOULD offer a canonical mode that sorts object keys alphabetically (Unicode code point order) at all nesting levels.
- **Note:** Canonical mode is not required for general conformance. It is an optional serializer feature. This spec does not define a canonical TERSE wire format in this version; that is deferred to a future revision.

13.4 Denial of Service via Deep Nesting

A malicious TERSE document with deeply nested objects or schema arrays with very large row counts could cause stack overflows or excessive memory allocation in recursive parsers.

- **Recommendation:** Parsers SHOULD enforce a configurable maximum nesting depth (default: 64 levels) and a maximum document size, rejecting inputs that exceed these limits with a parse error.

Appendix A: Grammar (ABNF)

Grammar conforming to RFC 5234. Revision history: v0.1 — initial pseudo-ABNF; v0.2 — block-obj unified to { } delimiters, safe-id expanded with explicit safe-start rule supporting . / @ characters, primitives isolated as top-level rules; v0.3 — tab indentation explicitly forbidden, number grammar precedence over safe-id formalized.

```
; Document and Top-Level Structure
document    = *( comment / key-value )
key-value   = key ":" [SP / NEWLINE] value NEWLINE
key         = safe-id / quoted-str
value       = null / boolean / number / string /
              object / array / schema-arr

; Primitives (Compact Signaling)
null        = "~"
boolean     = "T" / "F"
number      = ["-"] int [frac] [exp]    ; JSON number grammar

; Strings and Identifiers
string      = safe-id / quoted-str
safe-start  = ALPHA / "_" / "." / "/"
safe-char   = ALPHA / DIGIT / "-" / "_" / "." / "/" / "@"
safe-id     = safe-start *safe-char
; safe-id MUST NOT equal "T", "F", or "~"
quoted-str  = DQUOTE *( unescaped / escaped ) DQUOTE

; Objects (Unified with {} delimiters)
object      = inline-obj / block-obj
inline-obj  = "{" *( key ":" value SP ) "}"   ; no mandatory SP after {
block-obj   = "{" NEWLINE 1*( indent key-value ) "}"

; Arrays and Tables
array       = inline-arr / block-arr
inline-arr  = "[" *( value SP ) "]"
block-arr   = "[" NEWLINE 1*( indent value NEWLINE ) "]"
schema-arr  = "#[" 1*( safe-id SP ) "]" NEWLINE
              1*( indent 1*( value SP ) NEWLINE )

; Auxiliaries
indent      = 2SP                    ; mandatory 2-space indentation
comment     = "//" *( VCHAR / SP ) NEWLINE
```

```
NEWLINE     = %x0A                  ; LF only — CRLF normalized before
parse
SP          = %x20
; Note: WSP (%x09 tab) is explicitly FORBIDDEN — see §3.2
; ALPHA in this grammar = ASCII A-Z / a-z only (U+0041-U+005A, U+0061-
U+007A)
```

Appendix B: Reference Examples

B.1 Complete Application Configuration

```
name: my-app
version: 2.1.0
private: T
author: {name:Alice email:"alice@co.com"}
scripts: {dev:vite build:"vite build" test:vitest}
dependencies: {react:"^18.2.0" zustand:"^4.4.1"}
config:
  {
  port: 3000
  debug: F
  logLevel: warn
  tags: [web typescript spa]
  }
```

B.2 REST API Response — Uniform Array

```
total: 5
page: 1
data:
  #[id name email role active score]
  1 "Ana Lima" ana@co.com admin T 98.5
  2 "Bruno Melo" bruno@co.com editor T 87.2
  3 "Carla Neves" carla@co.com viewer F 72.0
```

B.3 Deeply Nested Order

```
orderId: ORD-88421
status: confirmed
customer: {id:1042 name:"Rafael Torres" email:"r@email.com"}
shipping:
  {
  address: "Rua das Flores, 123"
  city: "São Paulo"
  method: express
  estimatedDays: 2
  }
items:
  #[sku name qty unitPrice]
  PRD-001 "Notebook Pro 15" 1 4599.90
  PRD-002 "Mouse Wireless" 2 149.90
```

```
payment:
  {
  method: credit_card
  installments: 12
  total: 4924.70
  }
```

B.4 Mixed Types and Edge Cases

```
// Reserved values as strings must be quoted
flag: T                   // boolean true
label: "T"               // string "T"
value: ~                  // null
literal: "~"             // string "~"

// Empty collections
emptyObj: {}
emptyArr: []

// Heterogeneous array (not a schema array)
mixed:
  [
  1
  hello
  T
  ~
  {x:1 y:2}
  ]
```

Appendix C: Changelog

Version	Date	Notes
0.1.0	March 2026	Initial draft. All sections open for review.
0.2.0	March 2026	Fixed ABNF grammar (block-obj delimiters, safe-id expansion). Added Section 13 Security Considerations.
0.3.0	March 2026	Tabs forbidden; 2-space indentation mandatory. Top-level = JSON object. Number precedence over safe-id. Schema array quoting strengthened.
0.6.0	March 2026	YAML added to all comparison tables (Sections 8 and 9). New Section 8.2 with direct YAML vs TERSE token comparison. Coverage table expanded to 5 columns. Token efficiency table expanded with YAML column. Reading guidance updated to explain YAML position relative to TERSE.